2024 Sep Oppe 1 - Set 1
Section 2 - problem 2

Upper Case Only Vowels

Write a program that takes a passage with n lines of text as input. For each line, convert all vowels (a, e, i, o, u) to uppercase and all other characters to lowercase.

NOTE: This is an I/O type question, you need to write the whole code for taking input and printing the output.

Input Format

- The first line contains an integer n, the number of lines.
- The next n lines contain the passage.

Output Format

- Print the passage with vowels in uppercase and all other characters in lowercase.

Hint

Use the upper and lower methods of str to convert characters as needed.

Example

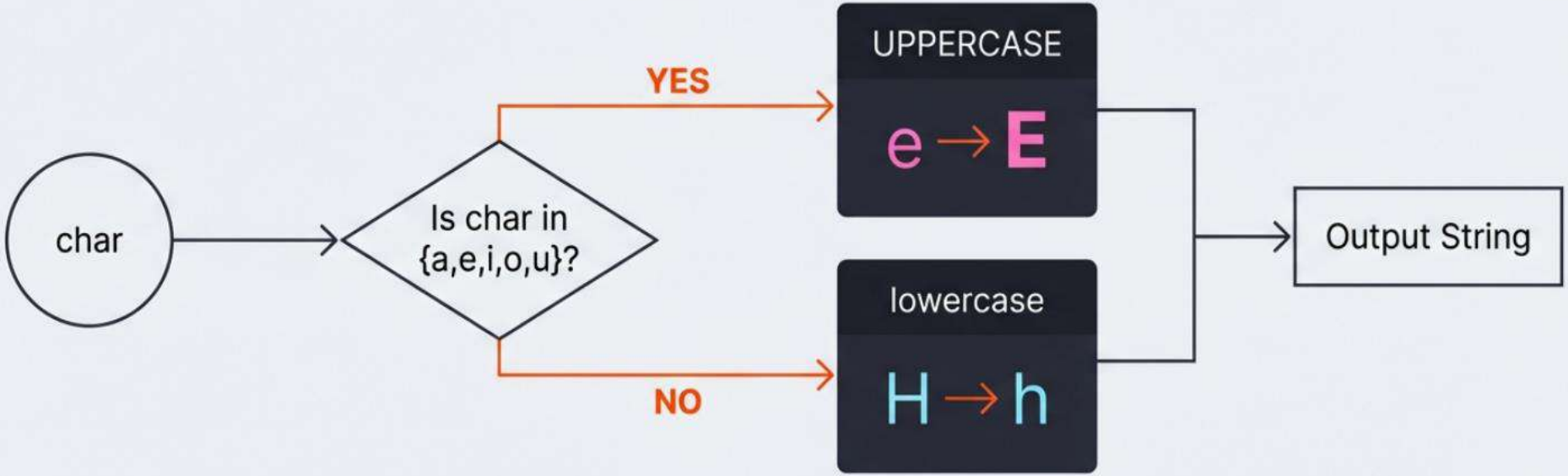
```
Input: 2
Hello World
Python Programming
```

```
Output: hEllO wOrld
pythOn prOgrAmmIng
```

The Specification

01. INPUT DATA	02. CONSTRAINTS	03. EXAMPLE
An integer ‘n’ followed by ‘n’ lines of raw string text.	If char is Vowel ({a, e, i, o, u}) convert to UPPERCASE. If Consonant or Symbol, convert to lowercase.	`Input: 'Hello World'` `Output: 'hEllO wOrld'`

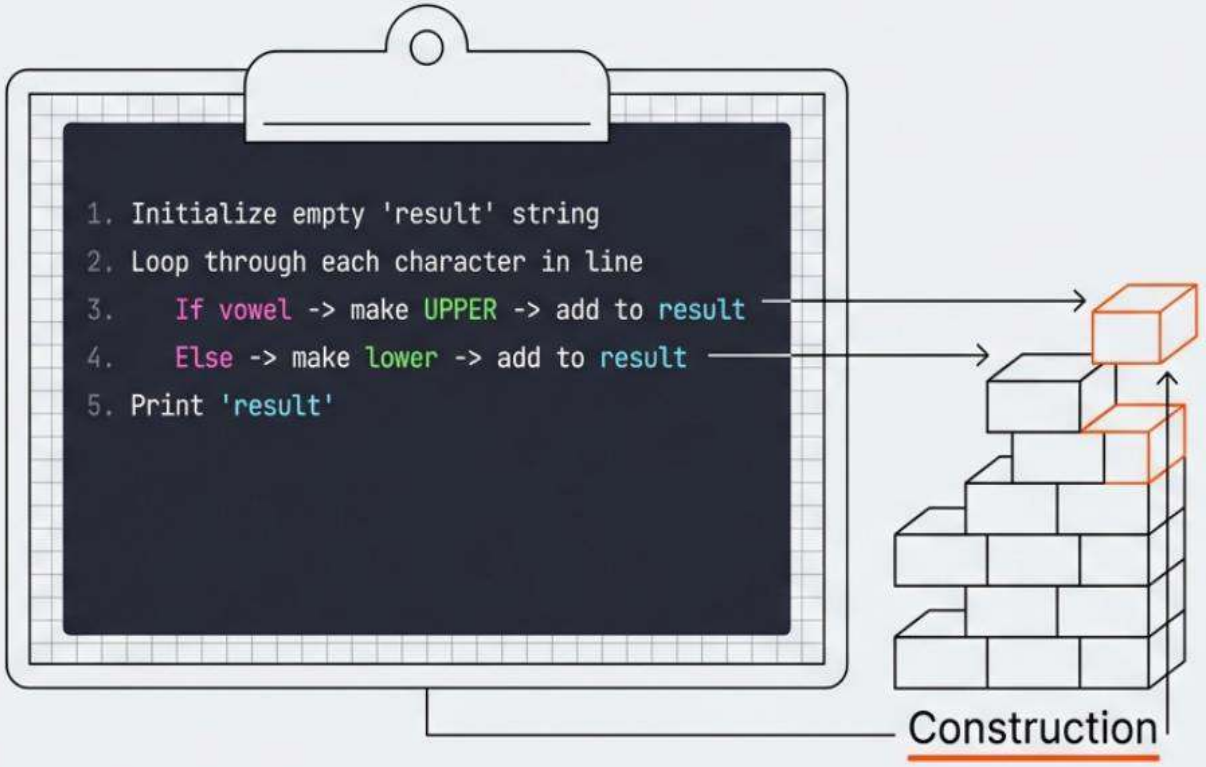
"Visualizing the Algorithm" with -1 tracking



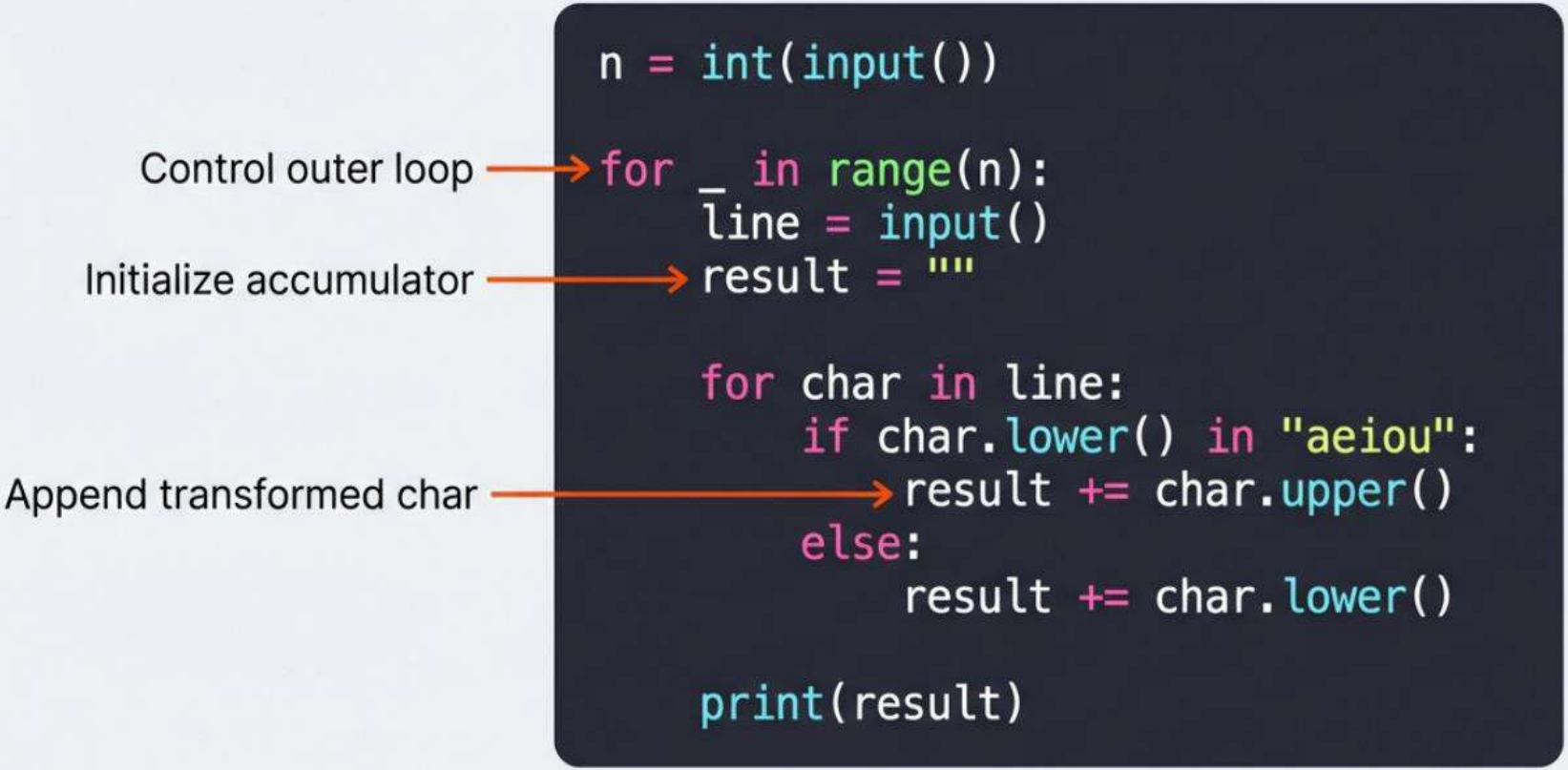
The logic requires treating the string as a sequence of individual character states.

Strategy 01: The Character Builder

The most direct way to solve this is by iterating through the string one character at a time and appending the result to a new container. This step-by-step accumulation allows for granular control.



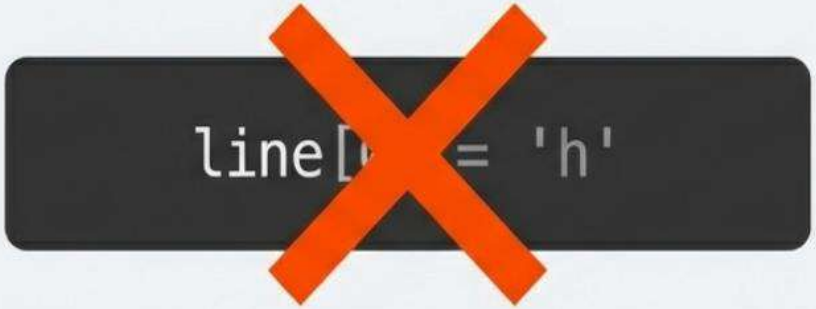
Implementation: The Manual Loop



Technical Constraint: String Immutability

Why we must build a new string instead of changing the old one.

The Wrong Way



TypeError: 'str' object does not support item assignment

The Right Way

Success: New String Object Created

Immutability forces the 'Builder Pattern'—we construct a completely new 'result' string rather than editing 'line'.

Strategy 02: The Pythonic Shift

Transforming the imperative loop into a functional expression.

```python
n = int(input())

for _ in range(n):
    line = input()
    result = ""

    for char in line:
        if char.lower() in "aeiou":
            result += char.upper()
        else:
            result += char.lower()

    print(result)
```

Compression

```
[ ...List Comprehension... ]
```

List Comprehension allows us to iterate and transform in a single step, while **.join()** handles the reconstruction.

Implementation: List Comprehension

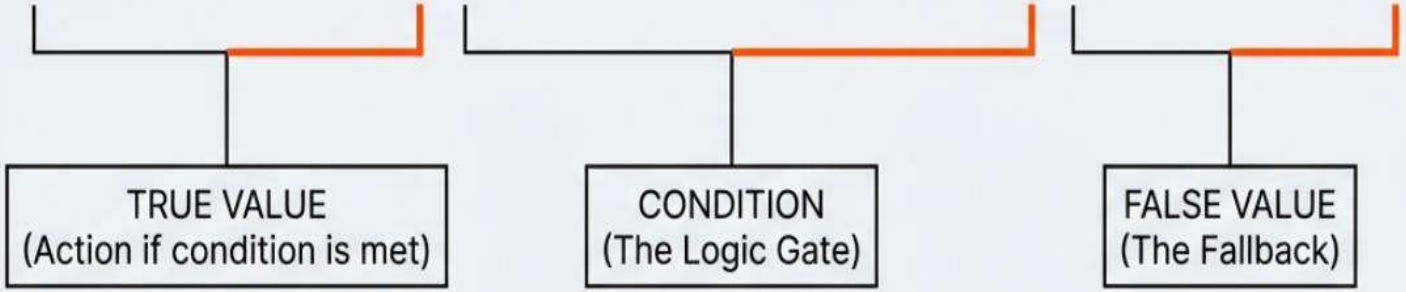
```
n = int(input())
vowels = "aeiouAEIOU"

for _ in range(n):
    line = input()
    transformed = "".join(c.upper() if c in vowels else c.lower() for c in line)
print(transformed)
```

The entire transformation logic is compressed into the generator expression inside the join method.

Syntax Zoom: The Ternary Operator

```
c.upper() if c in vowels else c.lower()
```



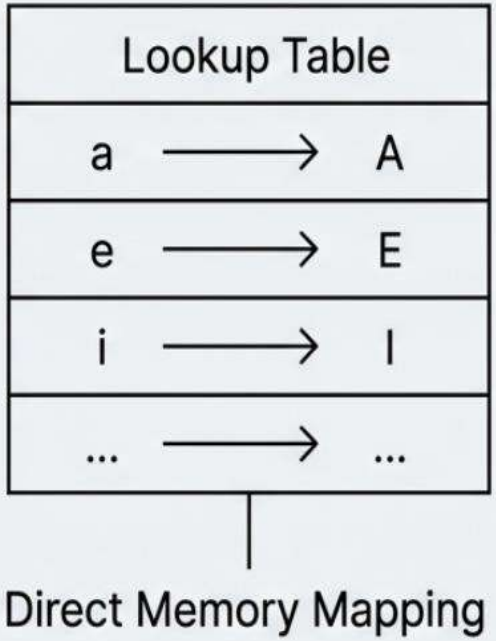
This syntax allows conditional logic to exist inline, removing the need for multi-line if/else blocks.

Strategy 03: Bulk Translation

For advanced use cases involving heavy text processing, Python offers a dedicated translation method using mapping tables.

```
# The Setup
trans_table = str.maketrans("aeiou", "AEIOU")

# The Execution
print(line.translate(trans_table))
```



Architectural Comparison

MANUAL LOOP	LIST COMPREHENSION Recommended	TRANSLATION
Logic Visibility: High (Explicit steps)	Logic Visibility: Medium (Inline)	Logic Visibility: Low (Abstracted)
Code Length: Verbose	Code Length: Concise	Code Length: Setup required
Use Case: Learning & Complex Logic	Use Case: Standard Transformations	Use Case: High-Speed Bulk Processing

Robust Implementation Details

Inter
Input Normalization

Always use `"int(input())"` for the **first line** to safely parse the loop counter `'n'`.

Inter
Case Insensitivity

Checking `"if char.lower() in vowels"` handles both `'A'` and `'a'` with a single check.

Inter
Mixed Input Handling

The `".lower()"` in the `else` clause ensures symbols and numbers pass through safely without crashing.

Inter
Robust Logic Flow

Key Takeaways

01 | RESPECT IMMUTABILITY

You cannot change a string in place. You must build a new one. The choice of method is simply a choice of how you build that new object.

02 | NORMALIZE FOR LOGIC

Simplify decision trees by checking against a normalized (lowercase) input, even if the output is mixed case.

03 | READABLE > CLEVER

While "maketrans" is fast, List Comprehension offers the best balance of elegance and maintainability for this task.